

Case study

PHẠM THỊ KIM NGÔN

Ngon.phamthikim@hoasen.edu.vn

Case study

1- Problem Description

2- Analysis

3- Design

4- Implementation

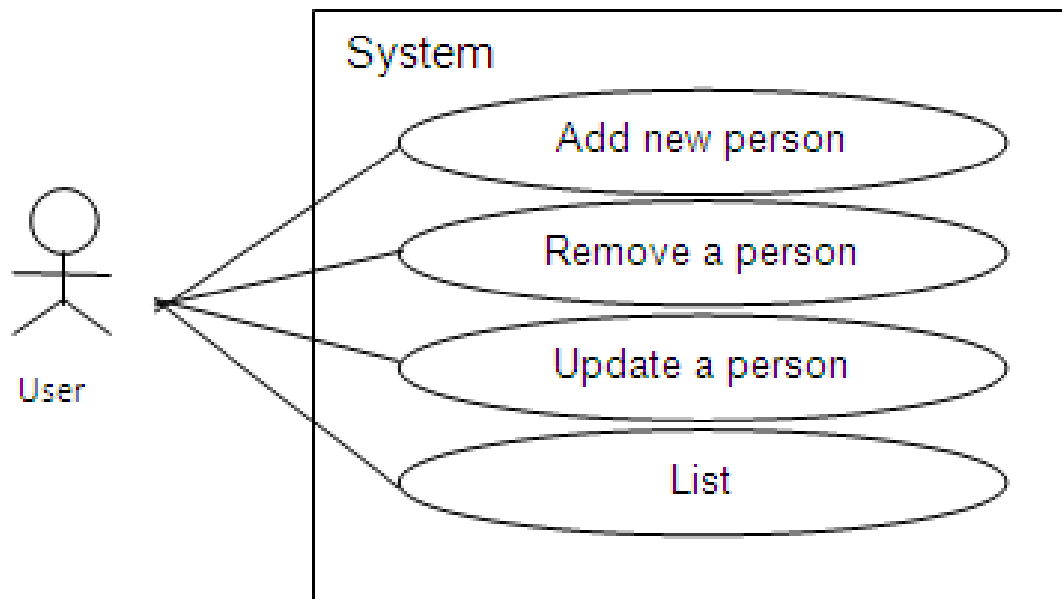
5- Testing

1- Problem Description

- Each person details include code, name, and age.
- Write a Java program that allows users adding a new person to the list, removing a person having a known code from the list, updating details of a known-code person, listing all managed persons in descending order of ages using a simple menu.

2- Analysis

From the problem description, following use-cases are identified:



- System/program is expressed as a bounded rectangle.
- Each function is expressed by a verb in an ellipse
- User runs a function is expressed as a line

3- Design

3.1- Class Design

From the problem description, concepts in the problem domain are expressed by following classes:

Class Person

Description for a person

Data: String code; String name; int age

Methods:

Constructors

Getters, setters

void input() for collecting data

String toString() to get data in string format

3.1- Class Design

Class PersonList

Description for a list of persons

Data:

Person[] list; // current list

int count // current number of persons

Methods:

Constructors

Getters, setters

void add(); // add a new person. Data are collected from keyboard

int find (String aCode); // Find the index of the person whose code is known

void remove();// remove a person. His/ her code is accepted from keyboard

void sort(); // descending sort the list based on their ages

void update(); // update a person, data are accepted from keyboard

void print(); // print the list

3.1- Class Design

Class Menu

Description for a menu

Data

String[] hints; // list of hints

int n; // current number of hints

Methods:

Menu(int n): constructor for initializing a menu containing n options

void add (String aHint); // add an option

int getChoice(); // get an option

Class ManagingProgram1

Description for the program

Data: none

Methods:

main(...): main method of the program

3.2- User interface

Menu of the program will be seen as:

1-Add new person

2-Remove a person

3-Update a person

4-List

5-Quit

4- Implementation

Initial data of the program (if any, file)

Please explore the software structure

Software

Please run the program

No.	Case	State
1	Add new person Code: not duplicate Name: Age:	Passed Passed not passed
2	Remove a person	Passed
3	Update a person	Passed
4	List	
...		

Recommendations

Code Conventions:

- **Indentation: 4 blanks at the beginning of each code line**
- **Comments in the code must be carried out.**
- **Names:**
 - **One-word name: lowercase**
 - **Multi-word name: The first word: lowercase, remaining words: The first character is uppercase, others are lowercase.**

Recommendations

A sample :

```
/*
```

```
Author: .....
```

```
Date: .....
```

```
This class represents .....
```

```
*/
```

```
class ClassName ..... {
```

```
    int data; // Which does data represent?
```

```
    ....
```

```
    /* What is the goal of the method
```

```
        Which does the return data represent?
```

```
    */
```

```
    Method implementation ..... {
```

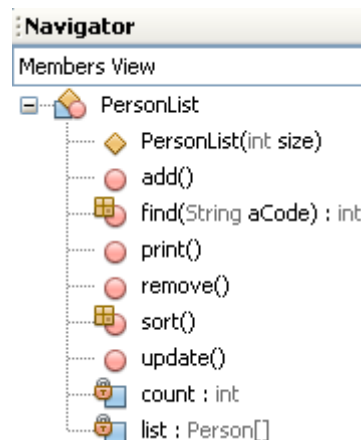
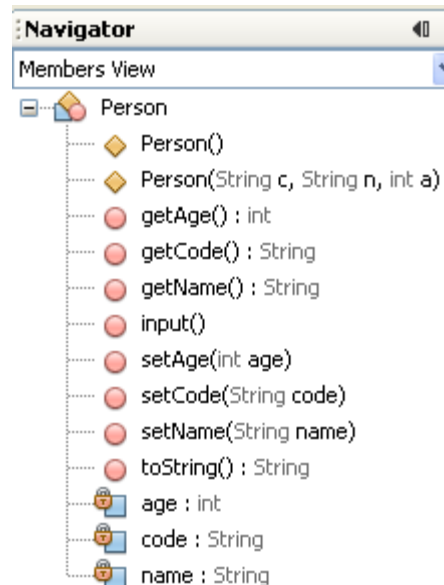
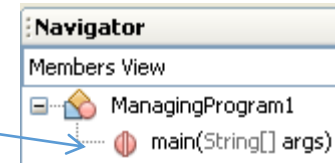
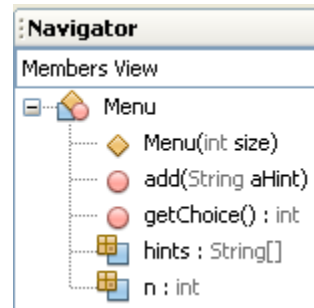
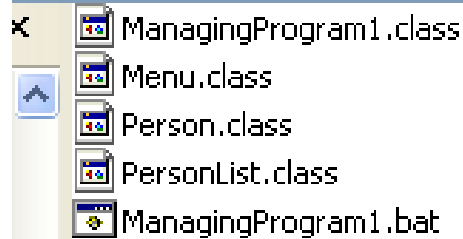
```
        ...
```

```
    }
```

```
}
```

Case study: Design Guide

(build)\classes



A screenshot of a Windows command prompt window titled "C:\WINDOWS\system32\cmd.exe". The window displays the output of the program, which is a text-based menu for a "PERSON MANAGER". The menu options are: 1-Add new person, 2-Remove a person, 3-Update a person, 4-List, and 5-Quit. The user has entered '1' to add a new person. The program then prompts for the person's code, name, and age. The user has entered 'NU01' for the code, 'Tuan' for the name, and '25' for the age. The program then displays the message "New person have been added." and shows the menu again. The user has entered a space character for the operation.

```
C:\WINDOWS\system32\cmd.exe

PERSON MANAGER
1-Add new person
2-Remove a person
3-Update a person
4-List
5-Quit
Please select an operation: 1
Enter the person's code: NU01
Enter the person's name: Tuan
Enter the person's age: 25
New person have been added.

PERSON MANAGER
1-Add new person
2-Remove as person
3-Update a person
4-List
5-Quit
Please select an operation: _
```

```

1  import java.util.Scanner;
2  public class Menu {
3      String[] hints;
4      int n = 0; // current number of hints
5      // create a menu with size elements
6      public Menu (int size)
7      {
8          if (size<1) size=10;
9          hints = new String[size];
10     }
11     // add a hint
12     public void add (String aHint)
13     {
14         if (n<hints.length)
15         {
16             hints[n++]=aHint;
17         }
18     }
19     // get user choice
20     public int getChoice()
21     {
22         int result=0;
23         if (n>0)
24         {
25             // print out hints
26             for (int i=0; i<n; i++)
27                 System.out.println( (i+1) + "-" + hints[i]);
28             System.out.print("Please select an operation: ");
29             Scanner sc= new Scanner(System.in);
30             result= Integer.parseInt(sc.nextLine()); // get user choice
31         }
32         return result;
33     }
34 }

```

Case study: Implementation

```
1  import java.util.Scanner;
2  public class Person {
3      private String code="", name=""; private int age=0;
4      // constructors
5      public Person() {}
6      public Person (String c, String n, int a)
7  □ { code=c; name=n; age=a>0? a: 0; }
8      // Getters and Setters
9      public String getCode() { return code; }
10     public void setCode(String code) { this.code = code;}
11     public String getName() { return name;}
12     public void setName(String name) { this.name = name;}
13     public int getAge() { return age; }
14     public void setAge(int age) { this.age = age;}
15     // Input details of the person
16     public void input()
17     { Scanner sc = new Scanner(System.in);
18       System.out.print("Enter the person's code: ");
19       code = sc.nextLine();
20       System.out.print("Enter the person's name: ");
21       name = sc.nextLine();
22       System.out.print("Enter the person's age: ");
23       age = Integer.parseInt(sc.nextLine());
24     }
25     // Method for output
26     🧠 public String toString()
27     { return code + ", " + name + ", " + age ;
28     }
29 }
```

Case study: Implementation

```
1  import java.util.Scanner;
2  public class PersonList {
3      private Person[] list= null;
4      private int count=0 ; // current number of persons
5      public PersonList( int size) // create a list with size persons
6      {
7          if (size<10) size=10;
8          list= new Person[size];
9      }
10     int find (String aCode) // find position of a known-code person
11     {
12         for (int i=0; i<count; i++)
13             if (aCode.equals(list[i].getCode())) return i;
14         return -1;
15     }
```

Case study: Implementation

```
public class PersonList
```

```
14     public void add()  
15 {   if (count == list.length) System.out.println("List is full!");  
16     else  
17     { String newCode, newName; int newAge;  
18         // Entering new person details  
19         Scanner sc= new Scanner(System.in);  
20         int pos; // variable for existing checking for new code  
21         do  
22         { System.out.print("Enter the person's code: ");  
23             newCode = sc.nextLine().toUpperCase();  
24             pos= find(newCode);  
25             if (pos>=0) System.out.println("\tThis code existed!");  
26         }  
27         while (pos>=0);  
28         System.out.print("Enter the person's name: ");  
29         newName = sc.nextLine().toUpperCase();  
30         System.out.print("Enter the person's age: ");  
31         newAge = Integer.parseInt(sc.nextLine());  
32         list[count++] = new Person(newCode, newName, newAge);  
33         System.out.println("New person have been added.");  
34     }  
35 }
```


Case study: Implementation

```
public class PersonList
```

```
36     public void remove()
37     {
38         if (count==0)
39         { System.out.println("Empty list.");
40             return;
41         }
42         String removedCode;
43         // Entering new person details
44         Scanner sc= new Scanner(System.in);
45         System.out.print("Enter the code of removed person: ");
46         removedCode = sc.nextLine().toUpperCase();
47         int pos = find (removedCode);
48         if (pos<0) System.out.println("This person does not exist.");
49         else
50         { // Shift up the remainder of the list
51             for (int i=pos; i<count-1; i++) list[i]= list[i+1];
52             count--;
53             System.out.println("The person " + removedCode + " was removed");
54         }
55     }
```

Case study: Implementation

`public class PersonList`

```
55     public void update() // updating name and age only
56     { if (count==0)
57         { System.out.println("Empty list.");
58           return;
59         }
60         String code;
61         // Entering the person's code
62         Scanner sc= new Scanner(System.in);
63         System.out.print("Enter the code of updated person: ");
64         code = sc.nextLine().toUpperCase();
65         int pos = find (code);
66         if (pos<0) System.out.println("This person does not exist.");
67         else
68         { // Update name and age
69           String newName; int newAge;
70           System.out.print("Enter the person's name: ");
71           newName = sc.nextLine().toUpperCase();
72           System.out.print("Enter the person's age: ");
73           newAge = Integer.parseInt(sc.nextLine());
74           list[pos].setName(newName);
75           list[pos].setAge(newAge);
76           System.out.println("The person " + code + " was updated");
77         }
78     }
```

Case study: Implementation

```
public class PersonList
```

```
79     public void print()
80     { if (count==0)
81         { System.out.println("Empty list");
82           return;
83         }
84         System.out.println("LIST OF PERSONS");
85         for (int i=0; i<count; i++)
86             System.out.println(list[i].toString());
87     }
88     void sort()
89     { if (count==0) return;
90       // Bubble Sort based on person's age
91       for (int i=0; i<count-1; i++)
92           for (int j=count-1; j>i; j--)
93               if (list[j].getAge()>list[j-1].getAge())
94                   { Person p = list[j];
95                     list[j]=list[j-1];
96                     list[j-1]=p;
97                   }
98     }
99 }
```

```
public class ManagingProgram1 {
```

```
    public static void main(String[] args)
    { Menu menu= new Menu(5);
      menu.add("Add new person");
      menu.add("Remove a person");
      menu.add("Update a person");
      menu.add("List");
      menu.add("Quit");
      int choice;
      PersonList list= new PersonList(50);
      do
      { System.out.println("\nPERSON MANAGER");
        choice=menu.getChoice();
        switch(choice)
        { case 1: list.add(); break;
          case 2: list.remove(); break;
          case 3: list.update(); break;
          case 4: list.sort(); list.print(); break;
        }
      } while (choice>=1 && choice <5);
    }
```

Summary

- The anatomy of a class, and how to declare fields, methods, and constructors.
- Hints for class design:
 - Main noun → Class
 - Descriptive nouns → Fields
 - Methods: Constructors, Getters, Setters, Normal methods
- Creating and using objects.
- To instantiate an object: Using appropriate constructor
- Use the dot operator to access the object's instance variables and methods.